

Competitive Programming

Section 24 Editorial: Grid DP

Competitive Programming Course

2026-07-18

Table of contents

1	Editorial overview	4
2	A. Safe Paths	5
2.1	Restatement	5
2.2	Constraints and traps	5
2.3	Observations	5
2.4	Sample walkthrough	5
2.5	Algorithm	6
2.6	Correctness proof	6
2.7	Complexity	6
2.8	Full C++ solution	6
2.9	Full Python solution	7
2.10	Common wrong answers	8
2.11	Tests that matter	8
3	B. Lowest Toll	9
3.1	Restatement	9
3.2	Constraints and traps	9
3.3	Observations	9
3.4	Sample walkthrough	9
3.5	Algorithm	10
3.6	Correctness proof	10
3.7	Complexity	10
3.8	Full C++ solution	10
3.9	Full Python solution	11
3.10	Common wrong answers	12
3.11	Tests that matter	12
4	C. Limited Turns	13
4.1	Restatement	13
4.2	Constraints and traps	13
4.3	Observations	13
4.4	Sample walkthrough	13
4.5	Algorithm	14
4.6	Correctness proof	14
4.7	Complexity	14
4.8	Full C++ solution	15
4.9	Full Python solution	16
4.10	Common wrong answers	17
4.11	Tests that matter	17
5	Testing and stress strategy	18
6	What to carry forward	19

7	D. Two Couriers	20
7.1	Restatement	20
7.2	Constraints and traps	20
7.3	Key observation and trace	20
7.4	State and algorithm	20
7.5	Correctness proof	21
7.6	Complexity	22
7.7	Full C++ solution	22
7.8	Full Python solution	22
7.9	Common wrong answers and tests	23
7.10	Stress testing	24
8	What to carry forward	25

1 Editorial overview

Problem	State	Transition
A. Safe Paths	ways to reach cell	sum from top and left
B. Lowest Toll	minimum cost to reach cell	toll + min(top, left)
C. Limited Turns	cell, turns, last direction	continue or turn
D. Two Couriers	common step and two rows	four pairs of next moves

The first three problems rely on the same row-major computation order. The third adds direction because the number of turns cannot be determined from the cell alone. The blind exercise instead synchronizes two routes at one step.

2 A. Safe Paths

2.1 Restatement

Given an $n \times m$ grid with open and blocked cells, count paths from the top-left to the bottom-right using only right and down moves. Paths may not enter blocked cells. Print the count modulo 1_000_000_007.

2.2 Constraints and traps

n and m can be up to 1000, so an $O(nm)$ DP is intended.

Important traps:

```
Blocked cells contribute zero paths.  
If the start or finish is blocked, the answer is zero.  
The answer can be huge, so take modulo after additions.  
Only right/down moves are allowed.
```

2.3 Observations

Every path reaching (r, c) must enter from:

```
(r-1, c) above  
(r, c-1) left
```

No other cell can be the immediate predecessor under right/down movement.

2.4 Sample walkthrough

Grid:

```
....  
.#..  
....
```

DP table:

```
1 1 1 1  
1 # 1 2  
1 1 2 4
```

Answer: 4.

2.5 Algorithm

```
if start is open:
    dp[0][0] = 1

for r from 0 to n-1:
    for c from 0 to m-1:
        if cell is blocked:
            dp[r][c] = 0
        else:
            add dp[r-1][c] if r > 0
            add dp[r][c-1] if c > 0

print dp[n-1][m-1]
```

2.6 Correctness proof

Lemma. For every open cell (r, c) , every valid path ending at (r, c) has immediate predecessor $(r-1, c)$ or $(r, c-1)$, unless (r, c) is the start.

Proof. The only allowed moves are down and right. Reversing the final move, a path into (r, c) must have come from one row above or one column left. $\square$

Theorem. The algorithm computes the number of valid paths to every cell.

Proof. Induct over row-major order. The start cell is initialized correctly. For any blocked cell, no valid path can stand there, so zero is correct. For an open non-start cell, the lemma partitions all paths by their final predecessor. Those sets are disjoint, and the predecessor values have already been computed in row-major order. The transition adds exactly their counts. $\square$

2.7 Complexity

```
Time:  $O(nm)$ 
Memory:  $O(nm)$ 
```

2.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    const long long MOD = 1000000007LL;
```

```

int n, m;
if (!(cin >> n >> m)) return 0;
vector<string> grid(n);
for (string &row : grid) cin >> row;
vector<vector<long long>> dp(n, vector<long long>(m, 0));
if (grid[0][0] == '.') dp[0][0] = 1;
for (int r = 0; r < n; r++) {
    for (int c = 0; c < m; c++) {
        if (grid[r][c] == '#') {
            dp[r][c] = 0;
            continue;
        }
        if (r) dp[r][c] = (dp[r][c] + dp[r - 1][c]) % MOD;
        if (c) dp[r][c] = (dp[r][c] + dp[r][c - 1]) % MOD;
    }
}
cout << dp[n - 1][m - 1] << '\n';
return 0;
}

```

2.9 Full Python solution

```

import sys

MOD = 1_000_000_007

def main() -> None:
    input = sys.stdin.readline
    first = input().split()
    if not first:
        return
    n, m = map(int, first)
    grid = [input().strip() for _ in range(n)]
    dp = [[0] * m for _ in range(n)]
    if grid[0][0] == ".":
        dp[0][0] = 1
    for r in range(n):
        for c in range(m):
            if grid[r][c] == "#":
                dp[r][c] = 0
                continue
            if r:
                dp[r][c] = (dp[r][c] + dp[r - 1][c]) % MOD
            if c:
                dp[r][c] = (dp[r][c] + dp[r][c - 1]) % MOD
    print(dp[n - 1][m - 1])

```

```
if __name__ == "__main__":  
    main()
```

2.10 Common wrong answers

Not zeroing blocked cells.
Letting paths pass through a blocked start or finish.
Forgetting modulo.
Using diagonal or upward moves.

2.11 Tests that matter

1x1 open grid.
Blocked finish.
A full open grid with many paths.
A wall that separates start from finish.

3 B. Lowest Toll

3.1 Restatement

Each grid cell has a nonnegative toll. Starting at the top-left and moving only right/down, reach the bottom-right with minimum total toll, including both start and finish cells.

3.2 Constraints and traps

$n, m \leq 1000$, so $O(nm)$ is intended.

Important traps:

```
The start cell's toll is included.  
The finish cell's toll is included.  
Tolls can be large, so use long long.  
Only the previous row is needed for memory.
```

3.3 Observations

The best path to (r, c) must come from the better of:

```
best path to  $(r-1, c)$   
best path to  $(r, c-1)$ 
```

Then add the current toll.

3.4 Sample walkthrough

Grid:

```
5 9 1  
4 2 7  
6 1 3
```

DP:

```
5 14 15  
9 11 18  
15 12 15
```

Answer: 15.

3.5 Algorithm

```
prev = INF row

for each row r:
    cur = INF row
    for each column c:
        if start:
            cur[c] = toll
        else:
            cur[c] = toll + min(prev[c], cur[c-1])
    prev = cur

print prev[m-1]
```

3.6 Correctness proof

Lemma. Any valid path to (r, c) must enter from above or left, except for the start.

Proof. The only moves are right and down. $\square$

Theorem. The algorithm computes the minimum cost to each processed cell.

Proof. Induct over row-major order. The start is initialized to its own toll. For any other cell, by the lemma, the final predecessor is above or left. By induction, `prev[c]` and `cur[c-1]` contain the optimal costs to those predecessors. Taking their minimum and adding the current toll gives the best possible path to (r, c) . $\square$

3.7 Complexity

```
Time:  $O(nm)$ 
Memory:  $O(m)$ 
```

3.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    const long long INF = (1LL << 62);
```

```

vector<long long> prev(m, INF), cur(m, INF);
for (int r = 0; r < n; r++) {
    fill(cur.begin(), cur.end(), INF);
    for (int c = 0; c < m; c++) {
        long long x;
        cin >> x;
        if (r == 0 && c == 0) {
            cur[c] = x;
        } else {
            long long best = prev[c];
            if (c) best = min(best, cur[c - 1]);
            cur[c] = best + x;
        }
    }
    swap(prev, cur);
}
cout << prev[m - 1] << '\n';
return 0;
}

```

3.9 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    m = next(it)
    inf = 10**30
    prev = [inf] * m
    for r in range(n):
        cur = [inf] * m
        for c in range(m):
            x = next(it)
            if r == 0 and c == 0:
                cur[c] = x
            else:
                best = prev[c]
                if c:
                    best = min(best, cur[c - 1])
                cur[c] = best + x
        prev = cur
    print(prev[-1])

```

```
if __name__ == "__main__":  
    main()
```

3.10 Common wrong answers

Forgetting to include the starting toll.
Using int for a path of many large tolls.
Using max instead of min after copying a counting recurrence.
Incorrect first row or first column initialization.

3.11 Tests that matter

1x1 grid.
One row.
One column.
Large tolls.
Best route intentionally avoids a locally cheap cell.

4 C. Limited Turns

4.1 Restatement

Count right/down paths through open cells from the top-left to the bottom-right that use at most K turns. A turn is a change between right and down on two consecutive moves. The first move is not a turn.

4.2 Constraints and traps

$n, m \leq 80$ and $K \leq 40$, so a state with cell, turn count, and direction is small enough.

Important traps:

```
dp[r][c] alone is not enough.  
The first move has no previous direction and uses zero turns.  
The 1x1 grid has one empty path.  
Blocked cells have zero states.  
The answer asks for at most  $K$  turns, so sum over  $0..K$ .
```

4.3 Observations

To decide whether entering (r, c) creates a turn, we must know the previous last direction. Therefore:

```
dp[r][c][t][dir]
```

where `dir` is the last move direction.

4.4 Sample walkthrough

For a 3×3 open grid with at most one turn, the only valid paths are:

```
RRDD  
DDRR
```

Every other right/down path changes direction at least twice. Answer: 2.

4.5 Algorithm

```
if start or finish is blocked: print 0
if n = 1 and m = 1: print 1

initialize first right move with 0 turns
initialize first down move with 0 turns

for every cell in row-major order:
    for turns from 0 to K:
        enter from left:
            continue RIGHT
            or turn from DOWN to RIGHT
        enter from above:
            continue DOWN
            or turn from RIGHT to DOWN

answer = sum both directions for turns 0..K at finish
```

4.6 Correctness proof

Lemma 1. Every non-empty path ending at a cell has exactly one last move direction: right or down.

Proof. The allowed moves are right and down. The final move, if it exists, is exactly one of them. $\square$

Lemma 2. The transition counts every valid path ending at (r, c) with exactly t turns and last direction dir once.

Proof. Consider $dir = RIGHT$. Such a path must enter from the left. Before the final move, it either already ended with $RIGHT$, adding no turn, or ended with $DOWN$, adding one turn. The transition includes exactly these two cases. They are disjoint by previous last direction. The $DOWN$ case is symmetric. $\square$

Theorem. The algorithm prints the number of valid paths with at most K turns.

Proof. The initialized first moves correctly represent one-move paths with zero turns. By induction over row-major order and Lemma 2, every DP state has the intended count. Summing the finish states over both last directions and turn counts $0..K$ gives exactly the requested paths. The 1×1 case is handled as the unique empty path. $\square$

4.7 Complexity

```
Time:  $O(nmK)$ 
Memory:  $O(nmK)$ 
```

The direction factor is constant.

4.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    const int MOD = 1000000007;
    const int RIGHT = 0, DOWN = 1;
    int n, m, K;
    if (!(cin >> n >> m >> K)) return 0;
    vector<string> grid(n);
    for (string &row : grid) cin >> row;
    if (grid[0][0] == '#' || grid[n - 1][m - 1] == '#') {
        cout << 0 << '\n';
        return 0;
    }
    if (n == 1 && m == 1) {
        cout << 1 << '\n';
        return 0;
    }
    vector dp(n, vector(m, vector(K + 1, array<int, 2>{0, 0})));
    if (m > 1 && grid[0][1] == '.') dp[0][1][0][RIGHT] = 1;
    if (n > 1 && grid[1][0] == '.') dp[1][0][0][DOWN] = 1;
    for (int r = 0; r < n; r++) {
        for (int c = 0; c < m; c++) {
            if (grid[r][c] == '#') continue;
            for (int turns = 0; turns <= K; turns++) {
                if (c > 0 && grid[r][c - 1] == '.') {
                    int val = dp[r][c][turns][RIGHT];
                    val += dp[r][c - 1][turns][RIGHT];
                    if (val >= MOD) val -= MOD;
                    if (turns) {
                        val += dp[r][c - 1][turns - 1][DOWN];
                        if (val >= MOD) val -= MOD;
                    }
                    dp[r][c][turns][RIGHT] = val;
                }
                if (r > 0 && grid[r - 1][c] == '.') {
                    int val = dp[r][c][turns][DOWN];
                    val += dp[r - 1][c][turns][DOWN];
                    if (val >= MOD) val -= MOD;
                    if (turns) {
                        val += dp[r - 1][c][turns - 1][RIGHT];
                        if (val >= MOD) val -= MOD;
                    }
                    dp[r][c][turns][DOWN] = val;
                }
            }
        }
    }
}
```

```

    }
}
}
int ans = 0;
for (int turns = 0; turns <= K; turns++) {
    ans += dp[n - 1][m - 1][turns][RIGHT];
    if (ans >= MOD) ans -= MOD;
    ans += dp[n - 1][m - 1][turns][DOWN];
    if (ans >= MOD) ans -= MOD;
}
cout << ans << '\n';
return 0;
}

```

4.9 Full Python solution

```

import sys

MOD = 1_000_000_007
RIGHT = 0
DOWN = 1

def main() -> None:
    input = sys.stdin.readline
    first = input().split()
    if not first:
        return
    n, m, k = map(int, first)
    grid = [input().strip() for _ in range(n)]
    if grid[0][0] == "#" or grid[n - 1][m - 1] == "#":
        print(0)
        return
    dp = [[[0, 0] for _ in range(k + 1)] for _ in range(m)] for _ in range(n)]
    if m > 1 and grid[0][1] == ".":
        dp[0][1][0][RIGHT] = 1
    if n > 1 and grid[1][0] == ".":
        dp[1][0][0][DOWN] = 1
    if n == 1 and m == 1:
        print(1)
        return
    for r in range(n):
        for c in range(m):
            if grid[r][c] == "#":
                continue
            for turns in range(k + 1):
                if c > 0 and grid[r][c - 1] == ".":

```

```

        dp[r][c][turns][RIGHT] = (
            dp[r][c][turns][RIGHT] + dp[r][c - 1][turns][RIGHT]
        ) % MOD
        if turns:
            dp[r][c][turns][RIGHT] = (
                dp[r][c][turns][RIGHT] + dp[r][c - 1][turns - 1][DOWN]
            ) % MOD
        if r > 0 and grid[r - 1][c] == ".":
            dp[r][c][turns][DOWN] = (
                dp[r][c][turns][DOWN] + dp[r - 1][c][turns][DOWN]
            ) % MOD
            if turns:
                dp[r][c][turns][DOWN] = (
                    dp[r][c][turns][DOWN] + dp[r - 1][c][turns - 1][RIGHT]
                ) % MOD

    ans = 0
    for turns in range(k + 1):
        ans = (ans + dp[n - 1][m - 1][turns][RIGHT] + dp[n - 1][m - 1][turns][DOWN]) % MOD
    print(ans)

if __name__ == "__main__":
    main()

```

4.10 Common wrong answers

Counting the first move as a turn.
 Forgetting to sum all turn counts up to K.
 Not storing last direction.
 Allowing paths to leave or enter blocked cells.
 Mishandling the 1x1 grid.

4.11 Tests that matter

1x1 grid.
 2x2 grid with K = 0 -> no path.
 Open 3x3 with K = 1 -> two paths.
 Blocked first right or first down cell.
 Large K should match ordinary path counting.

5 Testing and stress strategy

Run:

```
CP_TARGET=solution python3 sections/24_grid_dp/check.py
```

The random tests cover:

Safe Paths:

random blocked cells and zero-path grids.

Lowest Toll:

one-row, one-column, and random weighted grids.

Limited Turns:

small random grids with different K values and obstacles.

For tiny stress tests:

A: enumerate all right/down move strings for $n, m \leq 7$.

B: enumerate all right/down paths and take the minimum cost.

C: enumerate all right/down paths and count turns directly.

6 What to carry forward

Grid DP is about dependency direction.

```
If movement is acyclic, choose an order and fill a table.  
If a condition depends on history, add exactly the missing state.  
If only the previous row matters, row compression is available.
```

The next DP sections will use the same habits with intervals, trees, and bitmasks, where the correct order is less visually obvious.

7 D. Two Couriers

7.1 Restatement

Choose two right/down routes from the top-left to the bottom-right of a value grid. Their score is the sum of the values in the union of their visited cells: when both routes stand on one cell, its value is counted once. Maximize the score. Values may be negative, so a route cannot simply skip an inconvenient cell.

7.2 Constraints and traps

$n, m \leq 70$ makes an exponential search over two routes impossible. A route has $n+m-2$ moves, so even one route has a binomial number of possibilities.

```
Use 64-bit integers: as many as  $2 * (n+m-1)$  cells can contribute.  
Both couriers must make exactly  $n+m-2$  moves.  
The start and finish cells are shared and each contributes once.  
Do not independently optimize the two routes: their overlap changes the score.
```

7.3 Key observation and trace

After `step` moves, every right/down route is on a cell with:

```
row + column = step
```

Therefore, if we know courier one's row `r1` and courier two's row `r2`, their columns are forced:

```
c1 = step - r1  
c2 = step - r2
```

For example, after three moves, rows 1 and 2 mean positions (1,2) and (2,1). The state needs no separate columns. On the next move each courier chooses **right** or **down**, giving four transition pairs. Add both destination values, except add only one when the two destinations are identical.

7.4 State and algorithm

Let:

```
dp[r1][r2] = best score after `step` moves,
             with courier one at row r1 and courier two at row r2.
```

Initialize `dp[0][0]` with the start-cell value. For every step, create a new table initialized to negative infinity. For each reachable pair of rows, try the four pairs (right/right, right/down, down/right, down/down). Reject a move that leaves the grid. At `step = n+m-2`, both rows must be `n-1`.

```
dp[0][0] = value[0][0]

for step = 0 .. n+m-3:
    next = all negative infinity
    for every reachable r1, r2:
        derive c1 = step-r1 and c2 = step-r2
        for each of the four next-move pairs:
            derive nr1,nc1,nr2,nc2
            if both destinations are in the grid:
                gain = value[nr1][nc1]
                if destinations differ: gain += value[nr2][nc2]
                relax next[nr1][nr2]
    dp = next

answer = dp[n-1][n-1]
```

The two layers are sufficient because every transition advances exactly one step. This is also why negative infinity is necessary: zero is not a valid default when all available partial scores can be negative.

7.5 Correctness proof

Lemma 1. At a fixed `step`, rows `r1,r2` uniquely determine both courier positions whenever the derived columns are in range.

Proof. Each move increases `row + column` by one, starting from zero. Thus after `step` moves every position satisfies `row + column = step`; rearranging gives `column = step - row`. $\square$

Lemma 2. For every reachable state after `step` moves, `dp[r1][r2]` equals the maximum union score among all pairs of routes with those two positions.

Proof. Induct on `step`. At step zero there is only the pair of routes at the start, and the initialization counts that cell once. For the induction step, every pair of routes ending in a given next state has one of the four pairs of final moves. Removing those moves yields exactly a predecessor state considered by the algorithm. By induction its stored score is optimal; the transition adds precisely the new destination cells, once if equal. Taking the maximum over all four predecessor choices therefore stores the optimal score. $\square$

Theorem. The algorithm prints the maximum score obtainable by two legal courier routes.

Proof. After `n+m-2` moves, any legal courier is at the bottom-right cell, so all legal route pairs are represented by state `(n-1,n-1)`. Lemma 2 says that state stores the best score among exactly those pairs. $\square$

7.6 Complexity

There are $n+m-1$ layers and at most n^2 row pairs per layer. Each uses four constant-time transitions.

```
Time:     $O((n+m) \cdot n^2)$   
Memory:  $O(n^2)$ 
```

7.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    if (!(cin >> n >> m)) return 0;
    vector<vector<long long>> a(n, vector<long long>(m));
    for (auto &row : a) for (long long &x : row) cin >> x;

    const long long NEG = -(1LL << 62);
    vector<vector<long long>> dp(n, vector<long long>(n, NEG));
    dp[0][0] = a[0][0];
    for (int step = 0; step < n + m - 2; ++step) {
        vector<vector<long long>> nxt(n, vector<long long>(n, NEG));
        for (int r1 = 0; r1 < n; ++r1) for (int r2 = 0; r2 < n; ++r2) {
            int c1 = step - r1, c2 = step - r2;
            if (c1 < 0 || c1 >= m || c2 < 0 || c2 >= m || dp[r1][r2] == NEG) continue;
            for (int d1 = 0; d1 <= 1; ++d1) for (int d2 = 0; d2 <= 1; ++d2) {
                int nr1 = r1 + d1, nc1 = c1 + 1 - d1;
                int nr2 = r2 + d2, nc2 = c2 + 1 - d2;
                if (nr1 >= n || nr2 >= n || nc1 >= m || nc2 >= m) continue;
                long long gain = a[nr1][nc1];
                if (nr1 != nr2 || nc1 != nc2) gain += a[nr2][nc2];
                nxt[nr1][nr2] = max(nxt[nr1][nr2], dp[r1][r2] + gain);
            }
        }
        dp.swap(nxt);
    }
    cout << dp[n - 1][n - 1] << '\n';
}
```

7.8 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, m = data[:2]
    a = [data[2 + r * m:2 + (r + 1) * m] for r in range(n)]
    neg = -(1 << 62)
    dp = [[neg] * n for _ in range(n)]
    dp[0][0] = a[0][0]
    for step in range(n + m - 2):
        nxt = [[neg] * n for _ in range(n)]
        for r1 in range(n):
            c1 = step - r1
            if not 0 <= c1 < m:
                continue
            for r2 in range(n):
                c2 = step - r2
                if not 0 <= c2 < m or dp[r1][r2] == neg:
                    continue
                for d1 in (0, 1):
                    for d2 in (0, 1):
                        nr1, nc1 = r1 + d1, c1 + 1 - d1
                        nr2, nc2 = r2 + d2, c2 + 1 - d2
                        if nr1 >= n or nr2 >= n or nc1 >= m or nc2 >= m:
                            continue
                        gain = a[nr1][nc1]
                        if (nr1, nc1) != (nr2, nc2):
                            gain += a[nr2][nc2]
                        nxt[nr1][nr2] = max(nxt[nr1][nr2], dp[r1][r2] + gain)
        dp = nxt
    print(dp[n - 1][n - 1])

if __name__ == "__main__":
    main()

```

7.9 Common wrong answers and tests

Adding a meeting cell twice.
 Using 0 for an unreachable state on a negative grid.
 Forgetting that the 1x1 grid has only one counted cell.
 Trying to store both columns even though they are determined by the step.

Test a 1x1 negative grid, a single row or column where both routes must coincide, a grid where separating routes is valuable, and an all-negative grid. The random checker uses exhaustive

enumeration of every route pair on grids up to 5x5, then adds a 70x70 scale case.

7.10 Stress testing

For tiny $n, m \leq 5$, generate every right/down route by choosing the positions of its down moves. Try every pair of generated routes, form the union of their visited cells, and take the best value. This is deliberately a different algorithm from the synchronized DP, so agreement is meaningful. Include negative values in the tiny grids; they expose implementations that initialize unreachable states to zero. The separate 70x70 all-zero case checks that the intended polynomial state space is used without requiring an expensive oracle.

8 What to carry forward

Grid DP is not restricted to one path and one cell state. Find a quantity that advances in lockstep, then store only the independent coordinates. Here the step fixes both columns, reducing a four-coordinate description to two rows. The same synchronization idea reappears in problems with two walkers, aligned strings, and simultaneous operations.